Primordial Soup Replicator Simulation for Modeling Replication, Mutation, Stability, and Molecular Selection Dynamics

William Andrian Dharma T 13523006

Nathan Jovial Hartono 13523032

Abdullah Farhan 13523042

Muhammad Luqman Hakim 13523044

Institut Teknologi Bandung

Repository: https://github.com/kirisame-ame/Soupman

Video: videokds_soupman.mp4

ABSTRACT

This project presents a Python-based simulation of primordial soup replicators as an abstract computational model of early molecular evolution. Each replicator is represented as a discrete symbolic sequence with three heritable traits: replication rate, copying fidelity, and stability. The simulation investigates how these traits interact with stochastic mutation, resource limitation, lineage competition, and decay. The implementation uses a modular architecture consisting of a simulation environment, biological rule functions, a step-based simulation engine, a metrics tracker, and an interactive DearPyGui interface. The model shows that population survival is not determined by replication speed alone. High mutation increases sequence diversity but reduces average copying fidelity, low stability can drive the population to extinction, and resource regeneration expands carrying capacity while still preserving selection pressure. The resulting system demonstrates a dynamic balance between population growth, mutation-driven variation, structural persistence, and resource competition, making it a useful educational model for exploring fundamental principles of evolution and natural selection.

Index Terms — primordial soup, replicator, mutation, molecular evolution, natural selection, Python, DearPyGui, agent-based simulation.

1 INTRODUCTION

The concept of abiogenesis, or the emergence of life from non-living matter, has long been central to the study of the origin of life. One of the most fundamental theoretical frameworks is the "Primordial Soup" hypothesis, developed independently by Alexander Oparin and J.B.S. Haldane in the 1920s. This hypothesis suggests that the early conditions on Earth allowed for the formation of organic molecules in the oceans, creating an environment where the first

self-replicating molecules could emerge and initiate the process of molecular evolution.

In an effort to understand the dynamics of selection and competition during these early stages of molecular evolution, this project presents a Python-based simulation of primordial soup replicators as an abstract computational model. Each replicator is represented as a discrete symbolic sequence with three heritable traits: replication rate, copying fidelity, and stability. The simulation is designed to investigate how these traits interact with stochastic mutation, resource limitation, lineage competition, and decay.

Based on this framework, the main hypotheses are:

- The average total population of replicators will increase over time due to the availability of resources and their inherent replication ability (the expected, obvious outcome).
- The modeled environment, with its resource constraints and dynamic mutation mechanisms, can produce variations of replicators that employ different strategies (e.g., prioritizing stability or fidelity over replication rate) and are able to survive the competition, meaning the population will not be solely dominated by the fastest replicator.

2 Methodology

This section will cover the runtime algorithm in a high level and code analysis perspective. The explanation will be based off of the high level runtime flowchart in this link, the full source code can be found within the Github repository.

2.1 Initialization

The initialization phase consists of defining the configuration and then attaching said configuration to the engine and metrics tracker module for analysis.

The configuration such as initial population, resources, base fidelity, etc are necessary for both modules. The repository has provided four different lineage presets that focus either on high replication, fidelity, or stability and having the three balanced out. The configuration in detail can be found within the repository.

The engine is then initialized with the configuration determined at the start of the program, followed by the metrics tracker. After both engine and metrics tracker gets declared, the GUI module is also initialized with existing configuration and the two initialized modules.

The process continues with running the GUI module, where everything begins from setting up the required application context, building the UI window components, and setting up the viewport to mount the UI window components. We also set the metrics to track the simulation from the beginning timestamp, or default to timestamp = 0. The GUI is implemented using DearPyGui.

2.2 Application Loop Runtime

After the initialization process, the application's loop runtime begins by checking the application's GUI availability before proceeding. The application gets the delta time or dt between the previous and current time. The delta time is used to update the step's accumulator value.

This new value is then passed on to another loop that checks whether there is enough time left for the engine to take a step, by checking if the step accumulator value is still more than the interval then the loop will continue while also decrementing the step accumulator value. The loop proceeds with the engine "stepping" into a different time.

The engine will loop through all of the replicators in the engine's module environment attribute and apply rules determined by Random Number Generators whether it should decay, continue replicating, or do nothing. Obviously if it decays then we remove the replicator and refund the resources (resources is the value held by the environment that explicitly describes how much resources are left), if it continues replicating then we attempt replication by mutating the sequence and traits (replication rate, fidelity, stability) before creating the new Replicator object.

After updating the environment variables, the metrics will be updated with the updated env values computing values such as average replication rate, fidelity, and stability computing diversity using the env replicator values.

Finally the program will update the GUI with the updated engine environment and metrics value. The updates include updating particle states, drawing new particles, updating the lineage averages, and updating the plots. The program will execute back to the beginning of the application runtime.

2.3 Simulation Rules

The simulation uses discrete time steps. During each step, the engine processes all living replicators. First, the age of a replicator is increased. Then the system evaluates whether it decays. The probability of survival is computed from stability multiplied by the global stability multiplier. If the replicator dies, its sequence length is returned to the environment as resources.

$$P(replication) = clamp(replication\_rate \times replication\_multiplier)$$

If replication succeeds, the environment checks whether enough resources are available. The replication cost is equal to the length of the parent sequence. This means longer sequences are more expensive to reproduce. If resources are insufficient, reproduction fails even when the stochastic replication test succeeds.

$$P(mutation) = clamp((1 - fidelity) \times mutation\_multiplier)$$

Mutation occurs during copying. A lower fidelity value increases the probability that a copied symbol changes. The child also has a chance to experience trait mutation, which slightly changes replication rate, fidelity, and stability. After mutation, the traits are constrained by the total trait cap, stability cap, and stability-replication trade-off.

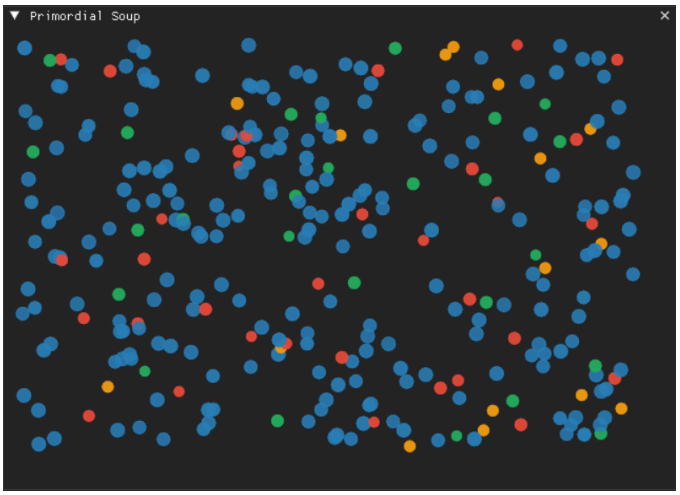

Figure 1. Simulation view of the primordial soup

3 Results and Discussion

Our experiments will be focused on seeing the end results of a determined time step limit regarding the population, diversity, global trait averages, and per lineage trait averages, over several initial environment conditions.

3.1 Setup

The default environment configuration has 2000 units of initial resources and a population of 80 with a ratio of 0.2, 0.2, 0.2, and 0.4 for the lineage presets listed below respectively. The initial lineage presets are as follows:

Table 1. Initial lineage presets

Lineage	Replication	Fidelity	Stability
High Replication	0.70	0.70	0.64
High Fidelity	0.34	0.96	0.88
High Stability	0.24	0.86	0.93
Balanced	0.32	0.88	0.90

We will test 200 time steps for 3 different environment configurations: default, resource abundant, and uncapped stability. The resource abundant configuration will have a high resource regeneration rate of 200 per step.

3.2 Results

Table 2. Average lineage population count by configuration

Lineage	Default	Resource Abundant	Uncapped Stability
High Replication	65.63	1433.53	58.80
High Fidelity	47.35	293.52	33.55
High Stability	33.70	190.15	91.17
Balanced	187.62	862.67	150.64
Total	334.3	2779.87	334.16

Table 3. Average trait value by configuration

Trait	Default	Resource Abundant	Uncapped Stability
Replication	0.238	0.307	0.201
Fidelity	0.853	0.797	0.846
Stability	0.879	0.853	0.909

In the default configuration, the Balanced lineage clearly dominates the average population (187.62), with High Replication and High Fidelity forming the next tier. This lines up with the global traits: replication and fidelity sit in the mid-range, and stability is moderately high. The system appears to favor the balanced trade-off strategy under baseline resource constraints, allowing steady growth without heavily skewing toward a single extreme trait.

In the resource-abundant setting, total populations explode across all lineages, but High Replication and Balanced surge the most (1433.53 and 862.67). The global trait averages shift in the same direction: replication rate rises (0.307) while fidelity and stability drop (0.797, 0.853). That pattern suggests abundant resources relax the selective pressure on accuracy and robustness, letting high-replication strategies capitalize on plentiful energy even if they are less stable or faithful.

With uncapped stability, the High Stability lineage jumps substantially (91.17) compared to default, and the global stability average is the highest of all configs (0.909). At the same time, replication is the lowest (0.201), indicating a trade-off: removing the cap pushes the population toward stability-heavy strategies at the cost of growth rate. Balanced still leads, but the system tilts toward persistence rather than rapid replication, which should also show up as steadier diversity dynamics over time.

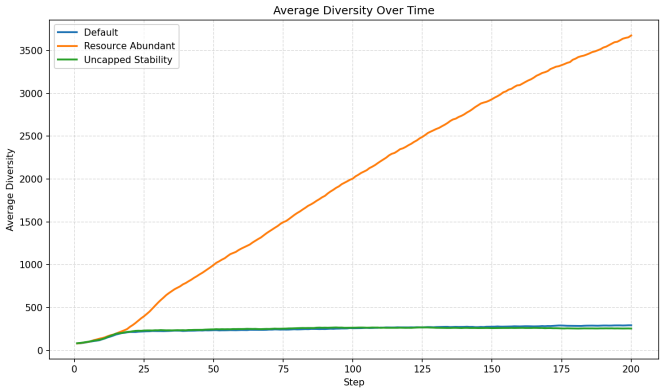

Figure 2. Average diversity over time for all three configurations

Across runs, it makes sense that one lineage usually becomes dominant: the simulation's fitness pressures are fairly direct (replication, stability, fidelity) and resource-limited, so any lineage that gets a slight early edge can compound it through higher replication or survival. That advantage snowballs because each successful replication reinforces the same trait bundle, and there are no strong counter-pressures to check the winner once it starts to pull ahead. Random drift at the start also matters: small early differences in births and deaths can lock in a trajectory that ends with a single clear leader.

Stable multi-lineage equilibria are rare because the model doesn't create sustained "niches" where different strategies are equally viable. Tradeoffs are static and global, so once the environment favors a trait profile, it keeps favoring it, and the population converges. Equilibrium would need either changing conditions or a negative feedback loop that penalizes the currently dominant strategy; without that, coexistence only happens when stochastic effects temporarily balance lineages.

Dawkins' arms-race framing hints at a natural fix: if "attack" and "defense" mechanisms co-evolved with explicit interactions, the fitness landscape would keep shifting. Introducing antagonistic dynamics, which can sustain diversity and prevent permanent dominance.

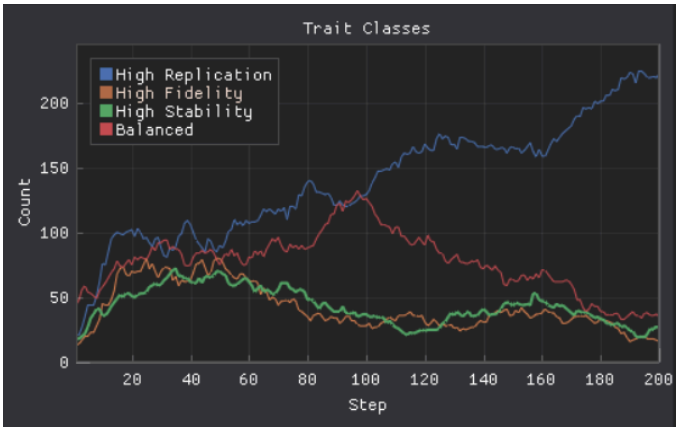

Figure 3. A run resulting in a single dominant lineage

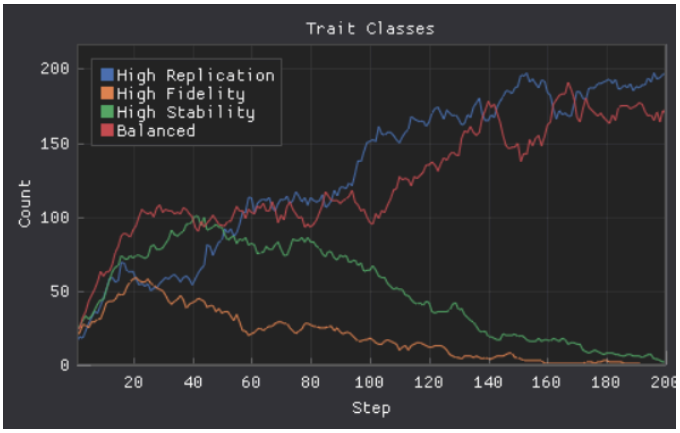

Figure 4. A run resulting in multi-lineage equilibrium

4 CONCLUSIONS

This simulation of primordial soup replicators demonstrates that even a minimal abstract model is sufficient to reproduce key dynamics theorized in early molecular evolution.

The first hypothesis was broadly confirmed: total populations grew across all three configurations, with the resource-abundant environment producing the most dramatic expansion. This validates the intuition that replicator abundance is fundamentally supply-constrained, and that releasing resource pressure unlocks rapid proliferation.

The second hypothesis received qualified support. Multiple lineages did persist across all configurations, and the dominance hierarchy shifted meaningfully depending on environmental conditions: the Balanced lineage led under default and uncapped stability settings, High Replication surged under resource abundance, and High Stability gained ground when its trait cap was removed. However, the simulation also revealed the limits of sustained coexistence. Once a lineage gained an early advantage under static fitness pressures, diversity tended to erode over time. True multi-lineage equilibrium was rare and transient, consistent with competitive exclusion in the absence of niche differentiation or negative frequency-dependence.

Together, these findings align with broader evolutionary theory. The model recapitulates the replication–fidelity–stability trade-off space central to origin-of-life research, and the diversity dynamics echo what would be expected from a Darwinian selection regime operating on heritable variation under resource constraint. The lack of sustained diversity suggests that introducing antagonistic co-evolutionary dynamics could sustain fitness landscape volatility and prevent permanent dominance.

Future work could incorporate spatially structured environments, dynamic resource cycles, or explicit molecular interaction networks to test whether richer ecological structure restores the multi-strategy equilibria this model only fleetingly achieves.

REFERENCES

[1] Dawkins, R. (2006). The selfish gene: 30th Anniversary edition. OUP Oxford.

[2] Schopf J. W. (2024). Pioneers of Origin of Life Studies-Darwin, Oparin, Haldane, Miller, Oró-And the Oldest Known Records of Life. Life (Basel, Switzerland), 14(10), 1345. https://doi.org/10.3390/life14101345

[3] Eigen, M. (1971). Selforganization of matter and the evolution of biological macromolecules. Die Naturwissenschaften, 58(10), 465–523. https://doi.org/10.1007/bf00623322

[4] Ray, T.S. (1991). An Approach to the Synthesis of Life.

[5] Ofria, C., Bryson, D. M., & Wilke, C. O. (2009). Avida: A Software Platform for Research in Computational Evolutionary Biology. Artificial Life Models in Software, 3–35. https://doi.org/10.1007/978-1-84882-285-6_1

[6] Wills, C. (2007). Principles of Population Genetics, 4th edition. Journal of Heredity, 98(4), 382. https://doi.org/10.1093/jhered/esm035